
SYSTEM ARCHITECTURE DOCUMENT

for

ShadePaths

Version 1.1

Prepared by Group A:
Raimundo Robredo Amaro
Ken Sunamoto
Mattu Yuvaraj Singh
Pyae Su Khaing
Suyama Riku
Yamaguchi Haruho

Made for PBL3
Ritsumeikan University

Last Modified on: 01/07/2026

Table of Contents

1	Introduction	3
1.1	Purpose	3
1.2	Intended Audience	3
2	Architectural Viewpoints	4
2.1	Deployment diagram	4
2.2	System Architecture diagram	5
2.3	System Flowchart	6

List of Figures

1	Deployment diagram	4
2	System Architecture diagram	5
3	System Flowchart	6

1. Introduction

1.1. Purpose

This System Architecture Document (SAD) describes the architectural decisions made for the ShadePaths system. It is intended to complement the SRS and SDD by explaining how and why the system is structured the way it is, and serves as a reference for the development team during implementation.

1.2. Intended Audience

This document is intended for the ShadePaths development team, the PBL course instructors who are in charge of reviewing it, and any future teams who may want to expand on the application.

2. Architectural Viewpoints

2.1. Deployment diagram

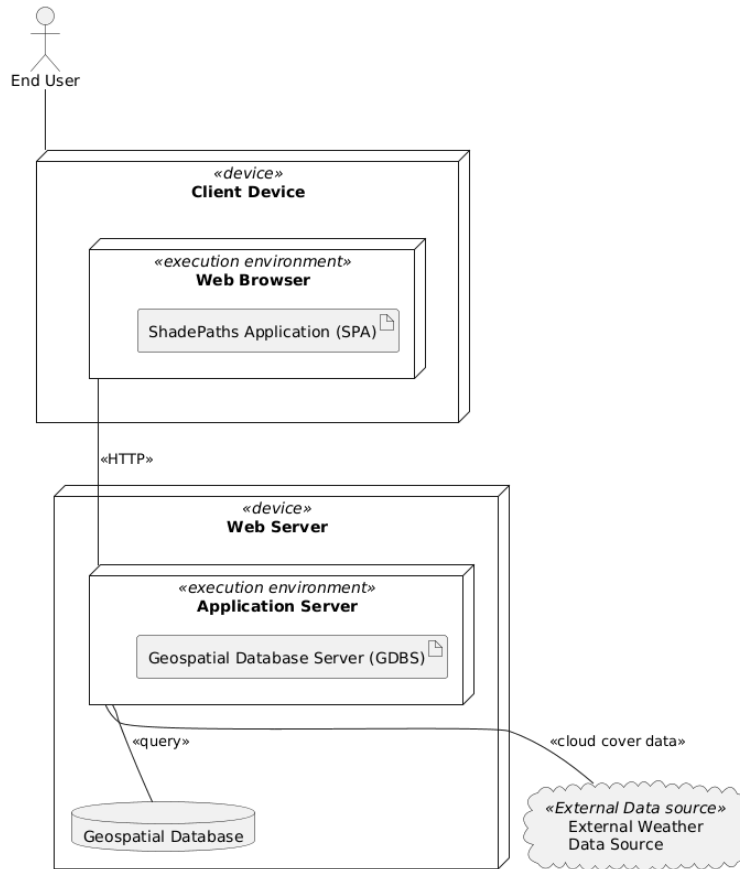


Figure 1: Deployment diagram

Description

The Client Device runs the SPA inside the Web Browser. The Web Server hosts two separate elements: an Application Server running GDBS, and the Geospatial Database as its own node, since the database is a separately running data store that GDBS depends on rather than part of the GDBS artifact itself. SPA and GDBS communicate over HTTP. GDBS queries the database directly and separately fetches cloud cover data from the External Weather Data Source, which sits outside the system boundary.

Rationale

The SRS only shows one undifferentiated server box, so this diagram had to decide whether GDBS and its database run as one deployed unit or two. They are split because the SDD's Class Diagram already models Geospatial Database Server as owning a database rather than being one, and the Component Diagram already shows Geospatial Database and Route Calculation as separate components within GDBS. Keeping them as separate nodes here makes that dependency explicit at the deployment level, and reflects that the database stays static after deployment while GDBS's weather data is updated periodically, two parts with different lifecycles that should not be modeled as a single artifact.

2.2. System Architecture diagram

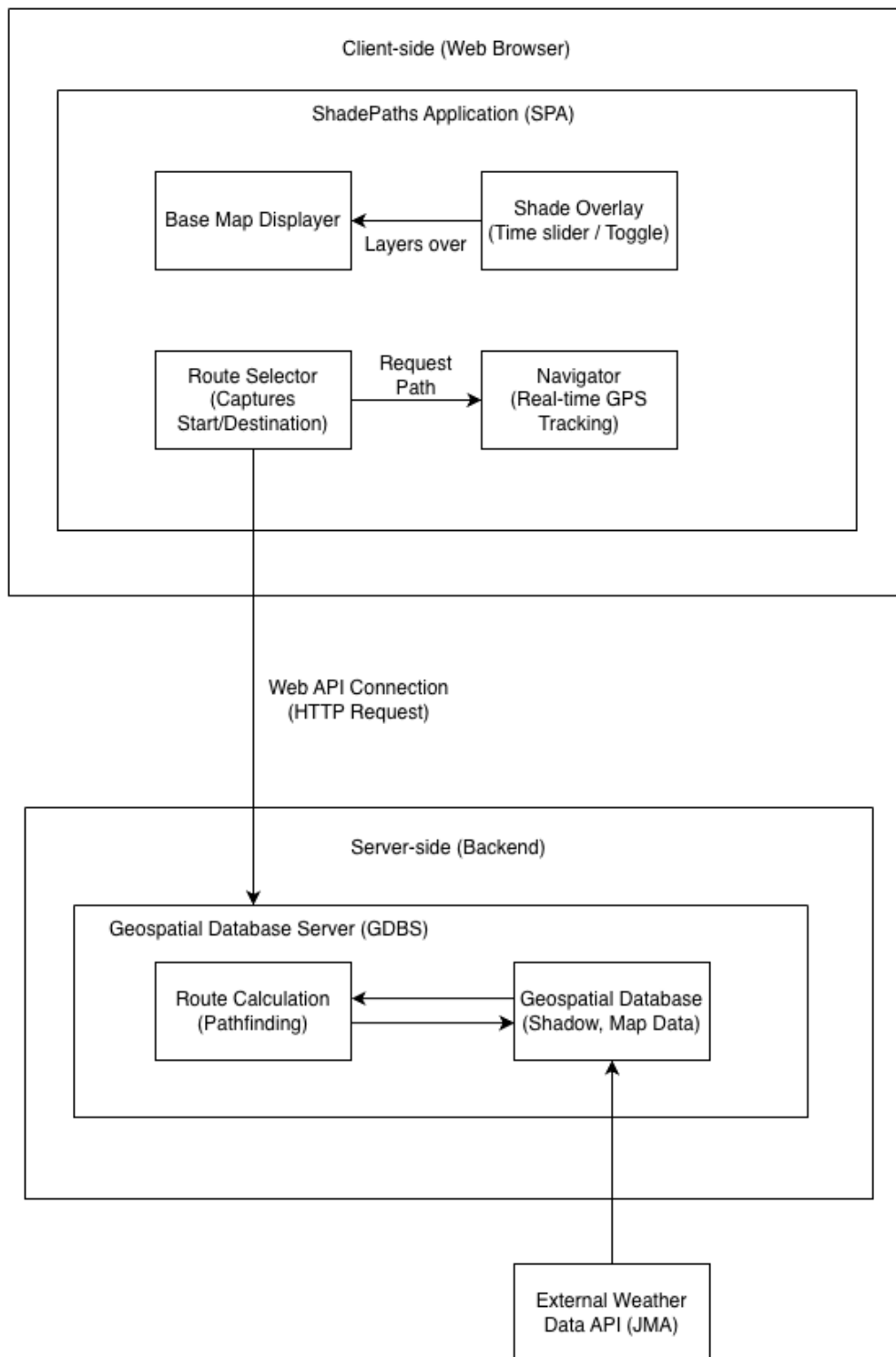


Figure 2: System Architecture diagram

Description

The system architecture diagram illustrates how ShadePaths is structured across two main layers. The client-side runs the ShadePaths Application (SPA) inside the web browser, which includes the Base Map Displayer, Shade Overlay controls, Route Selector for capturing start and destination inputs,

and the Navigator for real-time GPS tracking. The client sends HTTP requests to the server-side Geospatial Database Server (GDBS), which handles route calculation through its pathfinding logic and retrieves precomputed shadow and map data from the Geospatial Database. The GDBS also periodically fetches cloud cover data from the external JMA weather API to react in case the cloud cover surpasses the threshold specified in the SRS.

Rationale

This diagram is created to clarify where each main component runs and communicates with each other, which is Route Selector and Navigator run on the client while Route Calculation runs on the server. This helps clearly separate the frontend (UI) from the backend (server) which means frontend developers can focus purely on making a great user interface while backend developers can focus entirely on the logic and code.

2.3. System Flowchart

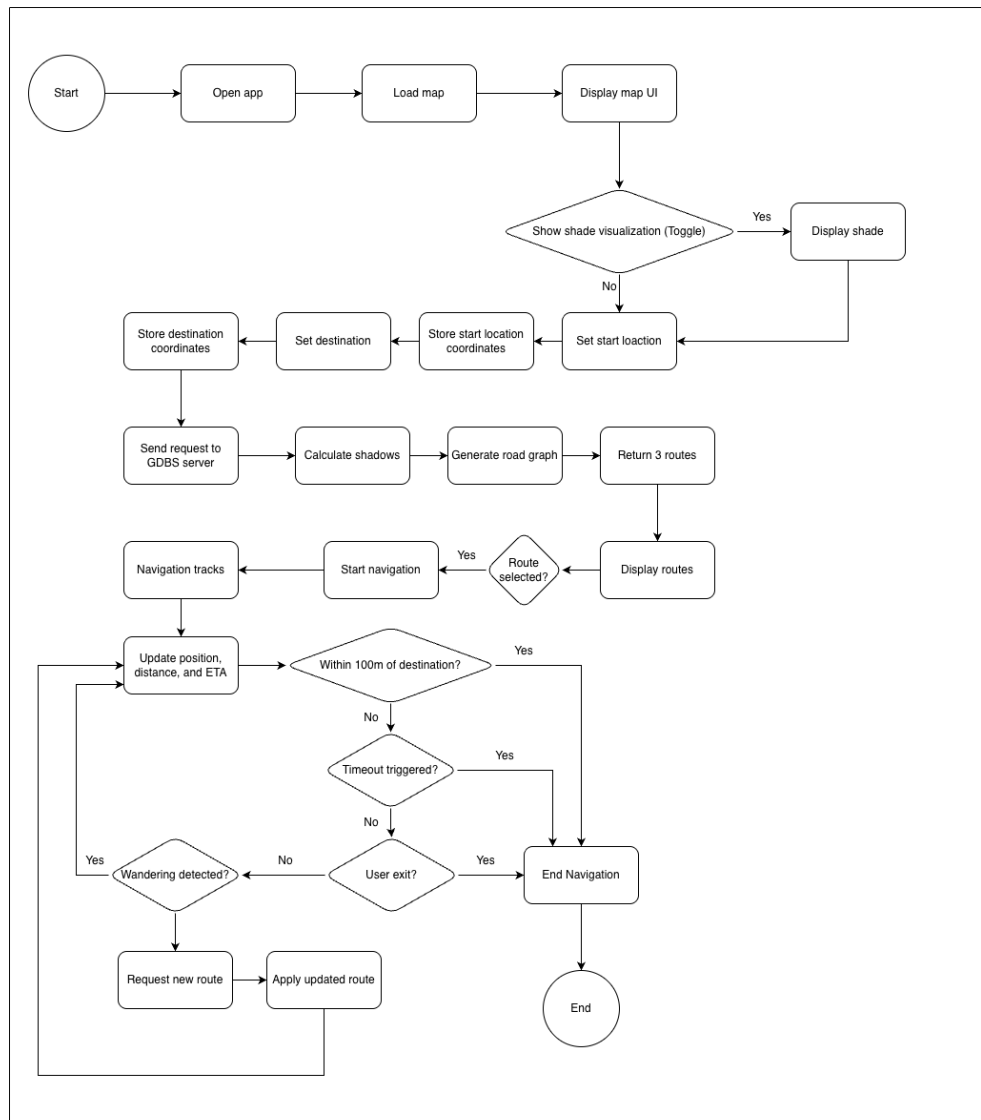


Figure 3: System Flowchart

Description

The flowchart maps out the step-by-step flow of the ShadePaths application from start to completion. It is worth noting that not every path shown will be taken in every session. For example, a user may open the app and go straight to entering a route without ever interacting with the shade toggle, and a navigation session may end cleanly without any wandering being detected. These optional branches are included regardless because the diagram is intended to give a complete picture of all the responses the system must be capable of exhibiting, which is more useful for the development team than a flowchart that only shows the minimal nominal execution path. As soon as the user opens the app, it loads the map interface with a toggle button allowing the user to view shade visualisation. Once the start and destination points are confirmed and stored, the client sends a request to the GDBS backend, where the server retrieves precomputed shadow data from the Geospatial Database, generates a road graph, and returns three route options for the user to select. After a route is chosen and navigation starts, the app updates the user's real-time position, distance, and ETA. The system then checks for exit conditions in order: reaching within 100m of the destination, a timeout, or a manual user exit. Optionally, if the user is detected wandering off the route, the system requests a new route from the GDBS, applies the update, and continues tracking.

Rationale

The flowchart is included to make the sequential logic of the application explicit before implementation begins. Rather than describing system behaviour in isolation as the SRS stimulus and response tables do, this diagram shows the order in which decisions are evaluated and how the system progresses from one step to the next. This is particularly useful for the navigation phase, where the order of exit condition checks matters for implementation. The optional wandering detection branch is included to reflect the OPT stimulus defined in SRS Table 10, keeping it visually distinct from the three required exit conditions.